

Control Statements

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1.	Introduction to Control Flow in Java.....	1
2.	Categories of Control Statements in Java	1
3.	Conditional Statements.....	5
4.	Difference between if-else and switch	8
5.	Summary of Key Concepts.....	8
6.	References.....	9

3.1.1 Introduction to Control Flow in Java

Control flow in Java refers to the order in which individual statements, instructions, or function calls are executed or evaluated. By default, Java executes code from top to bottom, one line at a time. However, control flow statements allow us to alter that default behavior, enabling decision-making, looping, and branching.

Why Control Flow is Important:

- To **make decisions** (if/else)
- To **repeat tasks** (loops)
- To **jump to specific parts** of code (branching)

3.1.2 Categories of Control Statements in Java

◆ 1. Conditional Statements

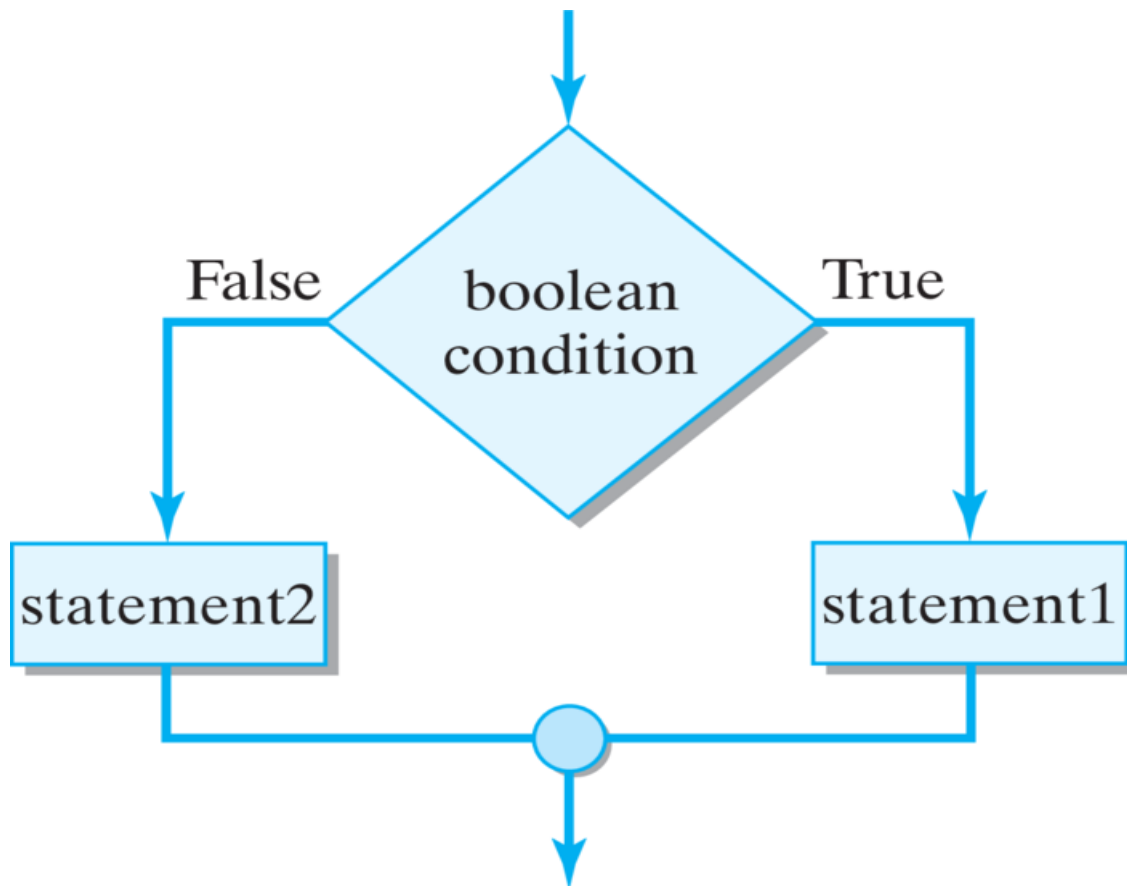
Purpose: Allow the program to make decisions and execute certain blocks of code based on specified conditions.

Types:

- if
- if-else
- if-else-if
- switch

Example:

```
int age = 18;
if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible to vote");
}
```



Conditional Statements: If - Else

Source: <https://cdn.programiz.com/sites/tutorial2program/files/java-for-loop.png>

◆ 2. Looping Statements

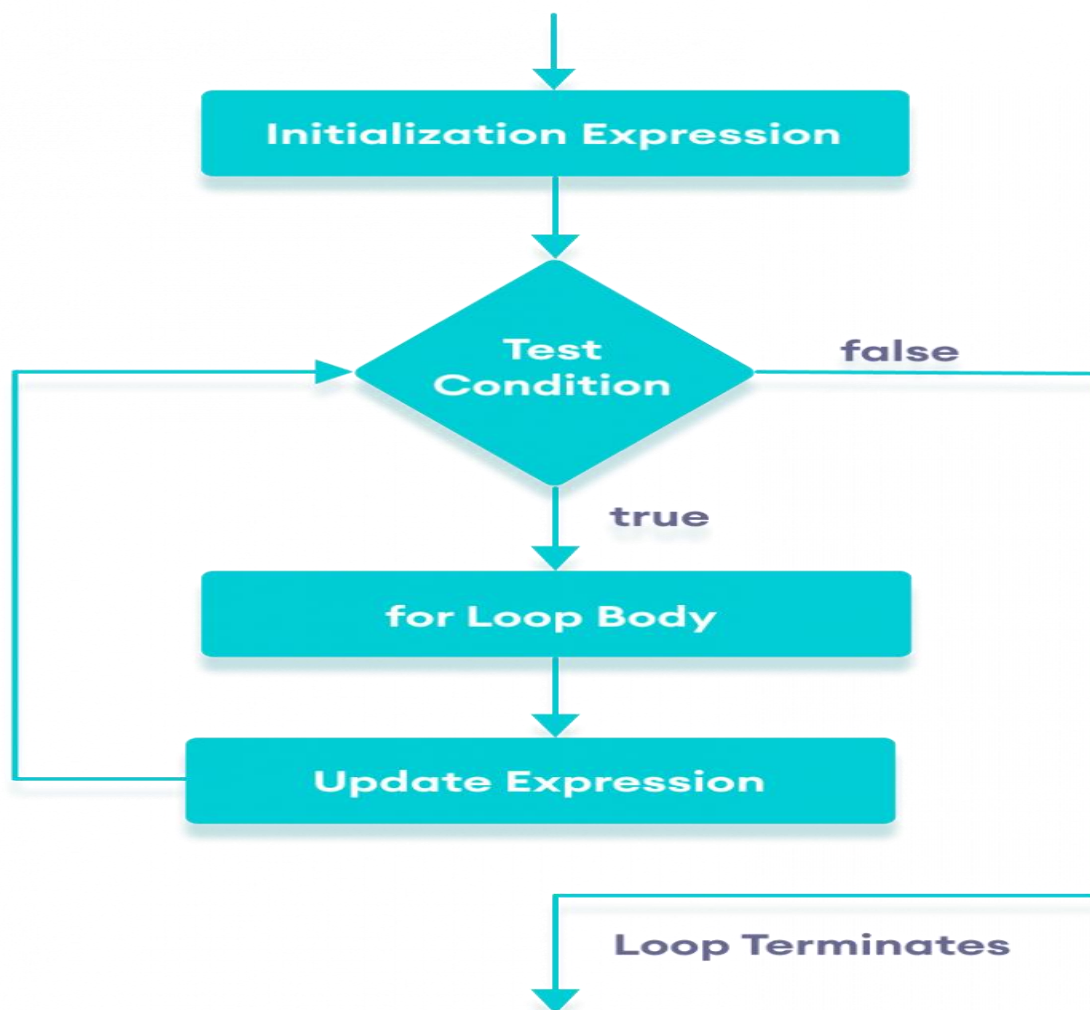
Purpose: Used to **repeat** a block of code **multiple times** as long as a specified condition is true.

Types:

- for
- while
- do-while

Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("Iteration " + i);  
}
```



Conditional Statements : For Loop

Source: <https://runestone.academy/ns/books/published/javajavajava/external/chptr03/3f14.png>

◆ 3. Jump Statements

Purpose:

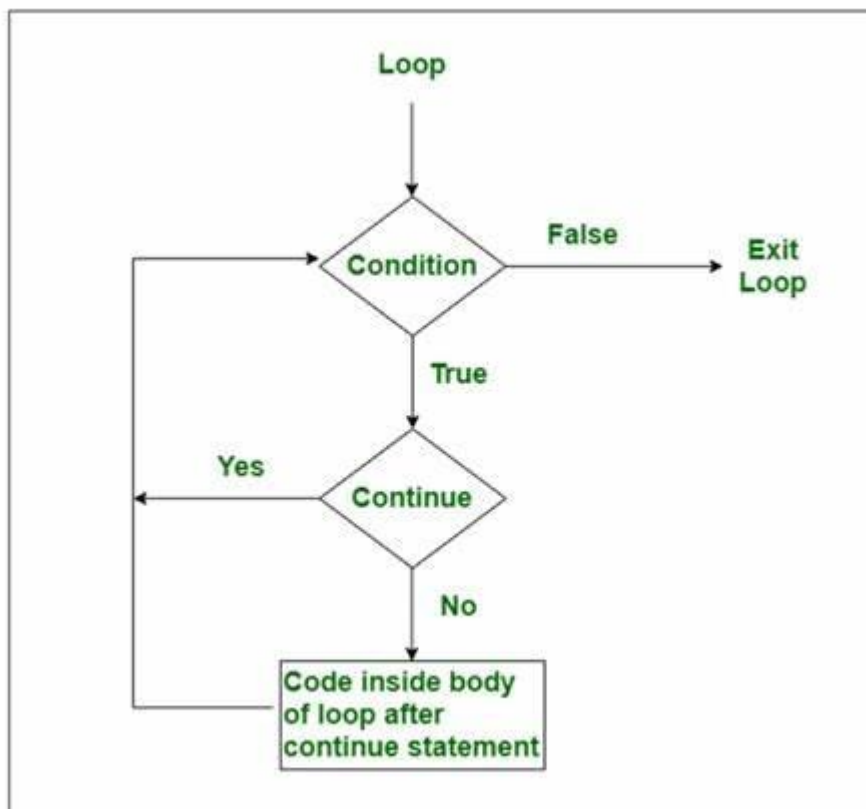
Provide control over the **flow of execution** by breaking or continuing loops, or returning from methods.

Types:

- break – exits the loop or switch statement.
- continue – skips the current iteration of a loop.
- return – exits from the current method.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // skip the 3rd iteration  
    }  
    System.out.println(i);  
}
```



Jump Statements: Continue Keyword

Source: <https://tse4.mm.bing.net/th/id/OIP.gbh4W-9ApnQ1Tb-1cxbLRwAAAA?rs=1&pid=ImgDetMain>

3.2.1. Conditional Statements

Conditional statements in Java allow programs to make decisions and perform different actions based on different conditions. These are essential in developing interactive and intelligent applications where the outcome depends on user input or other dynamic factors.

Types:

◆ 1. if Statement

- The simplest form of decision-making.
- Executes a block of code **only if the condition is true**.
- If the condition is false, the block is skipped.

Example:

```
int num = 10;
if (num > 5) {
    System.out.println("Number is greater than 5");
}
```

◆ 2. if-else Statement

- Provides two blocks of code: one if the condition is true, another if false.
- Offers a binary decision structure.

Example:

```
int age = 17;
if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible");
}
```

◆ 3. if-else-if Ladder

- Useful when there are **multiple conditions** to check.
- The conditions are evaluated from top to bottom until one is true.

Example:

```
int marks = 85;
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 80) {
    System.out.println("Grade B");
} else if (marks >= 70) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

◆ 4. Nested if Statements

- An if statement inside another if.
- Useful when a condition depends on another condition.

Example:

```
int num = 20;
if (num > 0) {
    if (num % 2 == 0) {
        System.out.println("Positive Even Number");
    }
}
```


◆ 5. switch Statement

- An alternative to long if-else-if chains when checking a single variable against multiple constants.
- Easier to read and more efficient in some scenarios.

Syntax:

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // code  
}
```

Example:

```
int day = 3;  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Other day");  
}
```

◆ **Important Notes about switch:**

- Accepts types: int, byte, short, char, String, and enums.
- Cannot use relational operators or complex conditions.
- break is essential to prevent fall-through (execution of subsequent cases).
- default is optional but recommended for handling unexpected input.

3.2.2. Difference between if-else and switch

Feature	if-else	switch
Data types	Works with all conditions	Limited to discrete values (int, char, etc.)
Complexity	Handles ranges and complex logic	Handles fixed values only
Use case	When logic is complex/multiple checks	When checking a variable against constants
Readability	Decreases with many conditions	Cleaner for fixed options
Performance	Slightly slower for large conditions	Potentially faster for simple constant values

➤ Summary of Key Concepts:

Conditional statements control the flow of execution based on conditions. Java supports:

- if – Executes a block if condition is true.
- if-else – Executes one block if true, another if false.
- if-else-if – Checks multiple conditions in sequence.
- Nested if – if inside another if for complex logic.
- switch – Selects code block based on variable's value; cleaner than multiple if-else for fixed options.

Use if for complex/logical checks, and switch for multi-way branching with constant values.

➤ References:

-  Oracle Java Documentation – Control Flow Statements
<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>
- GeeksforGeeks – Java Control Statements
<https://www.geeksforgeeks.org/control-statements-in-java/>
- W3Schools – Java Conditions and If Statements
https://www.w3schools.com/java/java_conditions.asp
- TutorialsPoint – Java Decision Making
https://www.tutorialspoint.com/java/java_decision_making.htm
- Java Complete Reference (Herbert Schildt) – Book
Especially Chapters on Control Flow and Conditional Statements

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>